Fault-Tolerant Data Archiving using Lempel-Ziv-Welch compression and Reed Solomon error correction codes

Adit Prabhu , Kartikya Saini - 24B4519, Darshan Jain

November 19, 2025

1 Introduction

In this project, we implemented a robust file archiving tool that combines lossless data compression with error correction. The system operates in two stages: first, it compresses data using the **Lempel–Ziv–Welch (LZW)** algorithm to reduce file size. Second, it applies **Reed-Solomon (RS)** coding to protect the compressed data against corruption.

Our primary goal was to understand the internal mechanics of these algorithms by implementing them from scratch in C++, rather than relying on existing libraries. We wanted to build a tool capable of compressing a file, surviving data corruption (like flipping bits via a hex editor), and then successfully recovering and decompressing the original file.

2 Part 1: Lempel-Ziv-Welch (LZW) Compression

2.1 Intuition

LZW is a lossless compression technique. It works by finding repeated patterns in the data and storing those patterns in a dictionary.

In simple terms:

- You start with a dictionary containing all 256 ASCII characters.
- As you read the file, you watch for repeated combinations like “AB”, “ABA”, “ABCABC”, etc.
- When a new pattern appears, you add it to the dictionary and assign it a unique

code number.

- Instead of storing the whole pattern, you store only the short code.
- During decompression, we rebuild the same dictionary in the same order and convert the codes back to the original sequences.
- You start with the default dictionary.
- Add the sequence corresponding to each code to the output data.
- Update the dictionary with the next code corresponding to the last character of the previous sequence concatenated with the current sequence
- There is one special case where the code we read might not yet exist in the dictionary. Although this seems strange, it is actually a normal part of the algorithm. If a code is not found in the dictionary, it represents: last output + first character of last output. This situation happens when the encoder creates a new pattern one step earlier than the decoder.

2.2 Example

To build intuition, we tried small examples ourselves.

Example

Input string: `ABABABA`

- First, only A and B are known.
- "AB" appears → dictionary adds "AB".
- Then "BA" appears → dictionary adds "BA".
- Then "ABA" appears → dictionary adds "ABA".

We realised that LZW keeps learning larger and larger patterns as long as they repeat. This makes it very efficient for data with repetitions.

Best Case vs Worst Case Compression Efficiency

- **Best Case:**
  Lots of repeated patterns → dictionary becomes powerful → high compression rate.
- **Worst Case:**
  Alphabet sequence:

ABCDEFGHIJKLMNOPQRSTUVWXYZ

Everything is unique → no useful patterns → almost no compression.
Other scenarios with low compression efficiency are when the data is random and contains no repeating patterns and when the data is already in compressed form (jpg, pdf, etc.)

2.3 Implementation of LZW

We divided the LZW implementation into two main parts.

(i). Compression Program

The compression program reads the input file, finds repeated patterns, assigns codes to them, and writes those codes as the compressed output.

Algorithm 1 LZW Compression

```
1: Initialize dictionary with all 256 ASCII characters
2: current_pattern ← empty string
3: for each character c in input do
4:     if (current_pattern + c) exists in dictionary then
5:         current_pattern ← current_pattern + c
6:     else
7:         output code for current_pattern
8:         add (current_pattern + c) to dictionary
9:         current_pattern ← c
10:    end if
11: end for
12: output code for current_pattern
```

(ii). Decompression Program

The decompression program reads the numeric codes and reconstructs the original file by rebuilding the same dictionary in the same order.

We implemented variable bit-width logic. We start with 9 bits (since 256 codes fit in 9 bits). When the dictionary grows to 512 entries, we switch to 10 bits, and so on, up to 12 bits.

Algorithm 2 LZW Decompression

```
Initialize dictionary with all 256 ASCII characters
read first_code
current_output ← dictionary[first_code]
write current_output
previous_output ← current_output
for each next_code in compressed data do
    if next_code exists in dictionary then
        current_output ← dictionary[next_code]
    else
                                                    ▷ Special case handling
        current_output ← previous_output + first_char(previous_output)
    end if
    write current_output
    add (previous_output + first_char(current_output)) to dictionary
    previous_output ← current_output
end for
```

2.4 Time Complexity

Even though compression depends on data patterns, the theoretical complexity of LZW is well studied.

Best Case Time Complexity:

$$O(n)$$

Dictionary lookups are fast because patterns repeat, and each step does almost constant work.

Worst Case Time Complexity:

$$O(n)$$

Even when no patterns repeat, LZW still processes each character once. The dictionary grows, but lookups remain average $O(1)$ when using hashing.

Overall Time Complexity:

$$O(n)$$

for both compression and decompression.

So overall, LZW remains linear-time in practice, regardless of repetition level.

3 Part 2: Reed-Solomon Error Correction

After compressing the data, we implemented Reed-Solomon (RS) coding to make the file robust against data corruption.

3.1 Galois Fields and Finite Arithmetic

In standard arithmetic, operations like division often result in fractions (e.g., $3/2 = 1.5$), and multiplication can result in very large numbers. However, digital data is stored in fixed-size bytes (0–255). We cannot have a byte overflow or turn into a fraction during error correction. Modular arithmetic does not work because integers modulo a number gives us a group and doesn't give us a field for composite numbers like 256. We require more properties related to multiplication for our purpose and specifically a well defined division operation, which requires a field.

To solve this, Reed-Solomon codes operate within a **Galois Field**, specifically $GF(2^8)$. A Galois Field is a finite set of elements where addition, subtraction, multiplication, and division are defined and the result of any operation stays within the set.

Representation: From Bytes to Polynomials

In this field, every 8-bit byte is not just a number, it represents a polynomial of degree 7 or less. The bits of the byte act as the coefficients (0 or 1) for powers of x.

For example, consider the decimal number 45:

$$45_{10} = (00101101)_2$$

This bit sequence corresponds to the polynomial:

$$0x^7 + 0x^6 + 1x^5 + 0x^4 + 1x^3 + 1x^2 + 0x^1 + 1x^0$$

$$= x^5 + x^3 + x^2 + 1$$

This relationship is fundamental because "adding" two numbers in this field is defined as adding their polynomials. Since coefficients are computed modulo 2 ($1 + 1 = 0$), polynomial addition is exactly equivalent to the bitwise XOR operation on the byte values.

Arithmetic Rules in $GF(2^8)$

- **Addition and Subtraction:** As established by the polynomial representation, these are identical and are performed using XOR.

$$5 + 6 = (00000101)_2 \oplus (00000110)_2 = (00000011)_2 = 3$$

- **Multiplication:** Elements are multiplied as polynomials. If the result exceeds degree 7 (values > 255), we divide by an **Irreducible Primitive Polynomial** and take the remainder. In our implementation, we used the standard primitive polynomial for QR codes:

$$P(x) = x^8 + x^4 + x^3 + x^2 + 1 \quad (0x11D)$$

- **Logarithms for Speed:** Performing polynomial multiplication and modulo for every byte is computationally expensive. We implemented Look-Up Tables for exponents and logarithms. This transforms complex multiplication into simple integer addition:

$$A \times B = \text{antilog}((\log(A) + \log(B)) \mod 255)$$

3.2 Intuition

The core intuition behind Reed-Solomon is the relationship between polynomials and their roots.

1. **Data as Polynomials:** We treat the message bytes as coefficients of a polynomial $M(x)$. For example, the data bytes $[d_2, d_1, d_0]$ become $d_2x^2 + d_1x + d_0$.

2. **Generator Polynomial:** The code is defined by a Generator Polynomial $G(x)$, which has specific roots (powers of a primitive element α).

$$G(x) = (x - \alpha^0)(x - \alpha^1)\dots(x - \alpha^{2t-1})$$

3. **The Key Property:** A valid codeword $C(x)$ is constructed such that it is exactly divisible by $G(x)$. Therefore, a valid codeword will evaluate to 0 at all the roots of $G(x)$. If we evaluate a received block at these roots and the result is not 0, we know an error has occurred.

Why Polynomials?

The core power of Reed-Solomon codes comes from a fundamental property of polynomials: **A polynomial of degree $k-1$ is uniquely defined by any k points.**

Think of this geometrically:

- Any 2 points define a unique line (degree 1).
- Any 3 points define a unique parabola (degree 2).

If we treat our data not as a list of numbers, but as points on a polynomial curve, we can add redundancy in a very powerful way. By adding extra points (parity bytes) that lie on the same curve, we create an "oversampled" system.

Controlled Redundancy and Recovery

If we have k data points and add $2t$ parity points, we transmit a total of n points. Even if noise corrupts some of these points (moving them "off" the curve), the original polynomial is still "locked in" by the remaining valid points.

Crucially, this protection is **distributed**. We do not protect specific "important" bits. Instead, the mathematical relationship binds all bytes together. As long as the total number of corrupted bytes is within the limit t, the algorithm can look at the noisy set of points, determine which curve fits the majority of them, and calculate exactly what the corrupted values should have been to lie on that curve.

Why is the redundancy double the number of corrections possible?

Suppose we have k bytes of data which we encode as an n bytes codeword out of which t bytes get corrupted. Now, the error in the recieved polynomial is defined by the difference between the recieved polynomial and the original codeword polynomial.

Since the recieved polynomial differs from the original polynomial at t bytes, the error polynomial consists of t terms. The recovery task consists of finding out the error polynomial from the recieved polynomial and subtracting it to get back the original data. The error polynomial which has t terms, consists of 2t unknown variables, which are, the location of error (power of x in each term) and the value of error (coefficient of each term).

Since there are 2t unknowns, given atleast 2t extra data points, we can form 2t equations and solve them to find out the error and correct it. If the number of extra data points are lesser than 2t, we will have more unknowns than equations and we will not be able to recover the original values. Hence, the amount of redundancy required is twice the number of errors we can correct.

Using $GF(2^8)$, we can handle error correction only on polynomials with degree $2^8 - 1 = 255$ or less. Hence, we split the files into blocks and apply error correction to each block.

For our implementation, we used an RS(255, 223) code. This means each block has 255 bytes total, containing 223 bytes of data and 32 bytes of parity. This configuration can correct up to $t = 16$ errors per $k = 223$ bytes.

3.3 Encoding Process

Encoding is the process of adding parity bytes to the message so that the resulting codeword is divisible by $G(x)$. We use **Systematic Encoding**, which means the original data remains visible in the first part of the codeword, followed by the parity.

To achieve this, we multiply the message polynomial $M(x)$ by x^{2t} (shifting it left to make room for parity) and then find the remainder when divided by $G(x)$.

$$C(x) = x^{2t}M(x) + (x^{2t}M(x) \bmod G(x))$$

Algorithm 3 Reed-Solomon Encoding

▷ All arithmetic on coefficients of the polynomials is done in $GF(2^8)$
$G(x) \leftarrow \prod_{i=0}^{31}(x - \alpha^i)$
▷ Appending 32 zeros corresponds to multiplying by x^{32}
$Padded_Msg \leftarrow$ Message + [32 Zeros]
$Remainder \leftarrow Padded_Msg$
for $i \leftarrow 0$ to $K - 1$ **do**
    $coef \leftarrow Remainder[i]$
    if $coef \neq 0$ **then**
        for $j \leftarrow 0$ to 31 **do** ▷ Subtract G(x) shifted by x^i
            $Remainder[i + j] \leftarrow Remainder[i + j] \oplus (coef \times G[j])$
        end for
    end if
end for
▷ Replace the placeholder zeros with the computed parity
$Codeword \leftarrow$ Padded_Msg + Last_32_Bytes($Remainder$)

3.4 Decoding Process

Decoding is significantly more complex. When we receive a block, it may contain errors: $R(x) = C(x) + E(x)$. We must identify $E(x)$ to remove it. The process involves four major steps.

3.4.1 Syndrome Calculation

We evaluate the received polynomial $R(x)$ at the roots of the generator polynomial $(\alpha^0, \ldots, \alpha^{31})$. Since $C(x)$ evaluates to 0 at these points, any non-zero result is due purely to the error $E(x)$.

$$S_i = R(\alpha^i) = C(\alpha^i) + E(\alpha^i) = 0 + E(\alpha^i)$$

Algorithm 4 Syndrome Calculation

$S \leftarrow$ Array of size $2t$
for $i \leftarrow 0$ to $2t - 1$ **do**
    $S[i] \leftarrow$ Evaluate $R(x)$ at α^i
end for
if all $S[i] == 0$ **then return** Success (No Errors)
end if

3.4.2 Berlekamp-Massey Algorithm

This is the heart of the decoder. We have a sequence of syndromes, and we need to find the "Error Locator Polynomial" $\Lambda(x)$. The roots of $\Lambda(x)$ will tell us the positions of the errors. The algorithm iteratively builds $\Lambda(x)$ to satisfy the known syndromes.

Algorithm 5 Berlekamp-Massey

$\Lambda(x) \leftarrow 1, B(x) \leftarrow 1$
$L \leftarrow 0$
for $n \leftarrow 0$ to $2t - 1$ **do**
    $discrepancy \leftarrow \sum_{i=0}^{L} \Lambda_i S_{n-i}$
    if $discrepancy \neq 0$ **then**
        $T(x) \leftarrow \Lambda(x)$
        $\Lambda(x) \leftarrow \Lambda(x) - discrepancy \times x \times B(x)$
        if $2L \leq n$ **then**
            $L \leftarrow n + 1 - L$
            $B(x) \leftarrow T(x)/discrepancy$
        end if
    end if
    $B(x) \leftarrow x \times B(x)$ ▷ Shift B for the next iteration
end for
return $\Lambda(x)$

3.4.3 Chien Search

Now that we have $\Lambda(x)$, we need to find its roots. Since the field is finite and small (256 elements), we can simply brute-force evaluate $\Lambda(x)$ at every possible position α^{-i}. If $\Lambda(\alpha^{-i}) = 0$, then there is an error at position i.

Algorithm 6 Chien Search

```
1: Error_Positions ← Empty List
2: for i ← 0 to 255 do
3:     X^{-1} ← α^{255−i}                    ▷ Corresponds to position i
4:     if Λ(X^{-1}) == 0 then
5:         Error_Positions.add(i)
6:     end if
7: end for
8: return Error_Positions
```

3.4.4 Forney Algorithm

Knowing the error positions is not enough; we need to know the error *value* (magnitude) to correct it. The Forney algorithm uses the syndromes and the derivative of the locator polynomial $\Lambda'(x)$ to calculate this value.

$$Y_i = X_i \frac{\Omega(X_i^{-1})}{\Lambda'(X_i^{-1})}$$

where $\Omega(x)$ is the Error Evaluator polynomial calculated from $S(x)$ and $\Lambda(x)$.

The formal derivative is an algebraic operation on polynomials that applies the power rule (nx^{n-1}) directly to coefficients without relying on limits or continuity, which don't exist in finite fields. Applying the power rule multiplies the coefficients by the scalar which is the power of x. Note that is the multiplication of elements of the Galois field with a scalar and not another field element. Since the Galois field consits of polynomials whose coefficients are integers modulo 2, multiplying any field element with an even scalar number makes all coefficients 0 and the result is 0 whereas multiplying any field element with an odd scalar keeps the coefficients the same which means the result is same as original element. This means the formal derivative of the error locator polynomial effectively just drops the even power terms and keeps the odd power terms the same.

Algorithm 7 Forney Algorithm

$\Omega(x) \leftarrow (S(x) \times \Lambda(x)) \pmod{x^{2t}}$
$\Lambda'(x) \leftarrow \text{FormalDerivative}(\Lambda(x))$ ▷ Drop all terms with even power
for each *pos* in *Error_Positions* **do**
    $X_{inv} \leftarrow \alpha^{-pos}$
    $numerator \leftarrow \text{Evaluate}(\Omega(x), X_{inv})$
    $denominator \leftarrow \text{Evaluate}(\Lambda'(x), X_{inv})$
    $magnitude \leftarrow (numerator/denominator)$
    $magnitude \leftarrow magnitude/X_{inv}$ ▷ Adjustment factor
    $Error_Magnitudes[pos] \leftarrow magnitude$
end for

3.5 Encoding Complexity

The encoding process consists of a polynomial division of the message polynomial $M(x)$ by the generator polynomial $G(x)$.

- **Per Block Complexity:** For every byte of the message (k bytes), we perform XOR and multiplication operations for each of the parity bytes ($2t$ bytes). Thus, the complexity per block is $\mathbf{O(k \cdot 2t)}$.
- **Total Complexity:** Since k and t are constants for the RS(255, 223) configuration, the encoding time scales linearly with the file size. Total complexity is $\mathbf{O(N)}$, where N is the total number of bytes.

Best Case: If the message data consists largely of zeros (sparse data), our implementation optimizes by skipping the subtraction step when the leading coefficient is zero. In the extreme case of an all-zero block, the complexity drops to $\mathbf{O(k)}$ per block (just scanning).

Time complexity with respect to the total size of the file, N is $\mathbf{O(N)}$ since there are $\frac{N}{k}$ blocks and each must be processed.

Worst Case: The message contains dense, non-zero data. Every byte triggers the polynomial subtraction loop. Complexity is $\mathbf{O(k \cdot t)}$ per block.

Time complexity with respect to the total size of the file, N is still $\mathbf{O(N)}$ since there are $\frac{N}{k}$ blocks and each must be processed.

3.6 Decoding Complexity

Decoding involves four distinct stages, each with different costs.

1. **Syndrome Calculation:** We evaluate the received polynomial (n bytes) at $2t$ roots. This takes $O(n \cdot 2t)$ operations. This step runs for *every* block.
2. **Berlekamp-Massey Algorithm:** This solves for the error locator polynomial. It runs for $2t$ iterations, with inner loops proportional to the number of errors found so far (up to t). Complexity is $O(t^2)$.
3. **Chien Search:** We evaluate the locator polynomial (degree $\leq t$) at all n field elements. Complexity is $O(n \cdot t)$.
4. **Forney Algorithm:** We calculate error magnitudes for e errors. Complexity is $O(e \cdot t)$.

Best Case (No Errors): If the file is clean, the Syndrome Calculation returns all zeros. The algorithm terminates immediately, skipping BM, Chien, and Forney.

$$\text{Best Case Complexity} \approx O(n \cdot t) \text{ per block}$$

(Dominated by Syndrome calculation).

Worst Case (Maximum Correctable Errors): If a block has $t = 16$ errors, all stages must execute. The Chien Search is typically the most expensive step because it requires evaluating a degree-16 polynomial 255 times.

$$\text{Worst Case Complexity} \approx O(n \cdot t + t^2) \approx O(n \cdot t) \text{ per block}$$

With respect to total file size (N): Since the file is divided into blocks, each of which has constant value of n and maximum possible errors corrected, t, the time complexity with respect to the file size is only proportional to no. of blocks, which is linear in the file size always.

$$\text{Time Complexity with respect to file size} = O(N)$$

Conclusion: While decoding is computationally more intensive than encoding (due to the syndrome and search steps), both operations remain linear $O(N)$ with respect to the total file size, making Reed-Solomon highly practical for large files.

4 Integration and System Architecture

Integrating LZW and Reed-Solomon presented a unique engineering challenge regarding data alignment.

The Mismatch:

- **LZW Output:** A stream of variable-width codes. It starts with 9-bit codes, increases to 10-bit, 11-bit, etc. This produces a bitstream that is not necessarily byte-aligned.
- **Reed-Solomon Input:** A stream of fixed 8-bit bytes grouped into blocks of 223.

Trying to pipe the live bitstream from LZW directly into the RS encoder logic would require complex bit-buffering and state management.

The Solution: Intermediate Storage To solve this, we decoupled the two stages using a temporary file system:

1. **Step 1 (LZW):** The raw input file is compressed. The variable-width codes are bit-packed and written sequentially to a temporary binary file (e.g., '.tmp.lzw'). This handles all bit-alignment and padding issues within the 'bitio' module.

2. **Step 2 (RS Encoding):** The RS tool reads this temporary file as a standard stream of bytes. It treats the compressed bitstream simply as "data," unaware that it represents variable-width integers.

3. **Step 3 (Framing):** We prepend a 4-byte header indicating the exact size of the compressed data. This is crucial because RS works in fixed blocks of 255 bytes, often requiring zero-padding at the end. Without the header, the decoder wouldn't know where the valid compressed data ends and the zero-padding begins.

Time Complexity
Since both LZW and Reed Solomon algorithms are linear in the size of the input file for both encoding and decoding, the overall time complexity of the program is **O(N)**

5 Results

Testing the program on range of inputs gave us:

- Python code in json format (.ipynb) - 182 $kb \rightarrow 50$ kb
- Pdf file (.pdf) - 2.0 $mb \rightarrow 2.4$ mb
- Image (.png) - 25 $kb \rightarrow 35$ kb
- Executable file (.out) - 72 $kb \rightarrow 45$ kb

Random edits were done using hexedit and the program was able to recover the original data back from the corrupted data. The program can fix upto 6.27 % of bytes being replaced as long as there are no more than 16 errors in a single block of 255.

We can see that compression works well on plaintext and raw data formats and struggles with more advanced formats which contain already compressed data.

6 References

- Welch, Terry. "A Technique for High-Performance Data Compression," IEEE, 1984.
- Reed, I. S., and Solomon, G. "Polynomial Codes Over Certain Finite Fields," SIAM Journal, 1960.
- Forney, G. David. "On Decoding BCH Codes," IEEE Transactions on Information Theory, 1965.
- Youtube